`sqisign-selkie`: Implementing SQIsign v2.0.1 NIST-I in Rust

Deirdre Connolly

Selkie Cryptography, USA

Abstract. SQIsign has the smallest signatures and public keys of any post-quantum signature scheme. It is also one of the hardest to implement correctly. We describe `sqisign-selkie`, a Rust implementation of SQIsign v2.0.1 for NIST-I, built from the specification and C reference implementation. Along the way we found nine bugs traceable to specification ambiguities or implicit C reference conventions, and four classes of bugs arising from fixed-precision arithmetic. We catalog these, explain how they arose, and extract concrete lessons for anyone implementing isogeny-based schemes from a spec. We show how Rust's type system enforces cryptographic invariants at compile time, survey the published constant-time techniques for SQIsign's quaternion arithmetic, and describe a negative test vector suite (in C2SP/wycheproof JSON format) that detects vulnerabilities—not just bugs—in SQIsign verification, signing, and key generation.

Keywords: SQIsign · isogeny-based cryptography · Rust · constant-time · post-quantum signatures

1 Introduction

SQIsign [SQI25] is the only isogeny-based signature scheme in Round 2 of NIST's additional signature standardization process. Its 148-byte signatures and 65-byte public keys at NIST-I are smaller than any other post-quantum scheme under consideration. But that compactness comes at a cost: implementing SQIsign requires quaternion algebras, the Deuring correspondence, dimension-2 theta-coordinate isogenies, and a signing algorithm that touches all of them.

The specification [SQI25] is mathematically precise but leaves many implementation choices unstated. The C reference implementation fills in those choices—but they are embedded in code, not documented. An implementor working from the spec alone will make wrong guesses. We know because we did.

This paper documents `sqisign-selkie`, a Rust library for the NIST-I parameter set ($p = 5 \cdot 2^{248} - 1$) only. By fixing the parameter set at compile time, we specialize aggressively: field arithmetic uses a radix-51 Montgomery representation tuned to this prime, integer widths match proven worst-case bounds for NIST-I quaternion intermediates [KLKL25], and isogeny degree types are sized to the 246-bit maximum that NIST-I permits. We do not attempt to be generic over parameter sets.

We target four goals:

1. **Interoperability.** Verify and produce signatures that the C reference implementation accepts, byte-for-byte, against the NIST KAT vectors.

E-mail: durumcrustulum@gmail.com (Deirdre Connolly)

2. **Misuse resistance.** If a value is always positive and odd, it should be impossible to construct one that isn't. If a lattice operation requires Hermite Normal Form, the compiler should reject non-HNF input. This protects not just end-users of the public API, but us writing the internals, and anyone who maintains this code after us.

3. **Constant-time signing (goal, not yet achieved).** The specification does not discuss side channels. The C reference is variable-time for the entire quaternion layer. We traced the signing algorithm and confirmed that the quaternion operations compute over secret-key-derived data. Published attacks [MCJ+25] demonstrate practical SPA on Cornacchia's algorithm. Constant-time execution is not a nice-to-have; it is a requirement. Our signing and key generation code is currently variable-time, with every secret-touching code path marked `TODO(ct)`. We plan to harden after achieving end-to-end interoperability (§7).

4. **Idiomatic Rust.** Methods on types over free functions. Invariants via distinct types, not runtime checks. Code that a Rust developer would recognize as natural [Rus24].

This paper is a living document, updated as the implementation progresses.

Contributions.

- A catalog of thirteen implementation bugs (§4, §5) with root causes and lessons.
- Compile-time invariant enforcement via Rust's type system (§6).
- A survey of constant-time SQIsign techniques and a concrete hardening plan (§7).
- A negative test vector suite for SQIsign verification, signing, and key generation (§8)—portable JSON vectors in C2SP/wycheproof format that other implementations can consume to detect vulnerabilities including exponent overflow, non-canonical field elements, degenerate kernels, and cross-field splicing attacks.

2 Background

We assume familiarity with elliptic curves and isogenies at the level of the SQIsign specification [SQI25], but not with the internals of theta-coordinate arithmetic or quaternion ideal algorithms. Where the spec is unclear, we say so; where the C reference makes an undocumented choice, we document it here.

3 Architecture

The implementation is organized into five layers:

`fields/` Prime field $\mathbb{F}_p$ (radix-51 Montgomery form) and quadratic extension $\mathbb{F}_{p^2}$.

`curves/` Montgomery curves, x-only projective arithmetic, torsion bases, 2^e-isogeny chains, and the `IsogenyDegree` newtype.

`surfaces/` Abelian surfaces in theta coordinates, $(2,2)$-isogeny chains (Algorithm 8.47), gluing, and splitting.

`quaternions/` Quaternion algebra $B_{p,\infty}$, lattices, ideals, HNF, L2 reduction, and `RepresentInteger`.

`deuring/` The Deuring correspondence bridge: `IdealToIsogeny`, `FixedDegreeIsogeny`, `SuitableIdeals`, action matrices.

3.1 Randomness and the `_derand` entry points

The SQIsign specification (Chapter 8) mandates **AES256-CTR-DRBG** (NIST SP 800-90A, §10.2.1) as the pseudo-random number generator for optimized implementations. The C reference follows this verbatim: its NIST PQC test harness (`PQCgenKAT_sign.c`) instantiates CTR_DRBG via `randombytes_init` with a 48-byte `entropy_input` and then services every call to `randombytes` from that state. Each KAT record in the `.rsp` files begins `seed = <96 hex chars>` – that 48-byte seed is the CTR_DRBG entropy input.

The 48 bytes are not arbitrary: for AES-256 the DRBG `seedlen` equals keylen + blocklen = 32+16 = 48. A shorter seed would leave part of the initial CTR_DRBG_Update state at zero and diverge from the NIST harness.

We expose two parallel APIs for keygen and signing:

- `SigningKey::generate<R: CryptoRngCore>(rng)` and `SigningKey::sign<R>(msg, rng)` sample 48 bytes of entropy from the caller's RNG and delegate to the derandomized entry points below. Production callers pass `OsRng`.

- `SigningKey::generate_derand(randomness: &[u8; 48])` and `SigningKey::sign_derand(msg, randomness)` take the 48-byte seed directly, instantiate a local `Aes256CtrDrbg` from it, and drive all internal sampling from that DRBG. These are the entry points that consume KAT seeds byte-for-byte.

Our `drbg` module implements AES256-CTR-DRBG directly on the RustCrypto `aes` crate, which provides AES-NI intrinsics on `x86_64` and ARMv8 cryptographic-extension intrinsics on `aarch64` out of the box. The module is a direct translation of the NIST reference (`CTR_DRBG_Instantiate`, `CTR_DRBG_Update`, `CTR_DRBG_Generate`): no derivation function, no reseed counter, no additional input. The DRBG is exposed as an `rand_core::RngCore`, so all generic sampling code in `quaternions::lattice` (rejection sampling, `rand_interval` inside `reduce_to_prime_norm`, `sample_from_ball`) drives it without further glue.

4 Bugs from Specification Ambiguity

During KAT verification testing against the C reference implementation, we discovered multiple bugs in our theta-coordinate $(2, 2)$-isogeny chain. Each bug originated from an ambiguity in the specification or an implicit convention in the reference code. We catalog them here with their root causes.

4.1 ProductToTheta coordinate convention

The spec describes the product theta structure abstractly via level-2 theta functions. The relationship between Montgomery projective coordinates $(X : Z)$ and the product theta representation is not stated explicitly. Our initial implementation used $(X - Z, X + Z)$ as "theta-like" coordinates—natural for Montgomery differential addition—but the C reference's `base_change()` uses raw (X, Z) directly:

```
fp2_mul(&null.x, &T->P1.x, &T->P2.x); // X1 * X2
fp2_mul(&null.y, &T->P1.x, &T->P2.z); // X1 * Z2
fp2_mul(&null.z, &T->P2.x, &T->P1.z); // Z1 * X2
fp2_mul(&null.t, &T->P1.z, &T->P2.z); // Z1 * Z2
```

Advice for implementers: When the spec is abstract about coordinate representations, the C reference is the ground truth.

4.2 Squared theta means square then Hadamard

In the SQIsign context, "squared theta" means component-wise squaring *followed by a Hadamard transform*. The C reference wraps this in `to_squared_theta()`, whose name does not reveal the Hadamard. Our generic isogeny evaluation (Algorithm 8.34) computed $\mathcal{H}(\text{inv} \cdot P^2)$ instead of $\mathcal{H}(\text{inv} \cdot \mathcal{H}(P^2))$.
Advice for implementers: Read the body of helper functions in the C reference, not just their names. "Squared theta" is not just squaring.

4.3 Cross-product index errors in GluingEval

Algorithm 8.39 computes `ADDComponents` on each curve component, producing (u_1, v_1, w_1) from curve 1 and (u_2, v_2, w_2) from curve 2. The U vector's second entry should be $u_1 \cdot w_2$ (cross-curve), but we wrote $u_1 \cdot w_1$ (same-curve).
Advice for implementers: In product-curve formulas, every multiplication involves one factor from each curve. Two factors with the same subscript are almost certainly wrong.

4.4 Projective inverse vs. field inversion

The gluing codomain needs the "inverse" of the dual theta null point $(\alpha, \beta, \gamma, 0)$. In projective coordinates this is computed via cross-products—pure multiplications, no field inversion. Our code called `Fp2::invert()`, which is both wrong and expensive.
Advice for implementers: In projective or theta coordinates, "inverse" almost always means projective inverse via cross-products. Actual field inversion is a red flag.

4.5 Reusing generic types for special patterns

The image of the kernel generator under gluing has the pattern $(x : x : y : y)$. Storing this in a generic 4-coordinate `JacobianPoint` creates ambiguity: `J.y` equals x (the second copy), not y. The scaling step used `J.x` and `J.y` for the two distinct values, but both hold x.
Advice for implementers: Do not reuse a generic n-coordinate type to store a value with a special pattern. Use a dedicated type, or at minimum document which fields hold which logical values.

4.6 Torsion basis swap convention

The C reference's `ec_curve_to_basis_2f_from_hint()` deliberately stores $P - Q$ in `B.Q` and Q in `B.PmQ`:

```
difference_point(&PQ2->Q, &P, &Q, curve);  // B.Q = P - Q
copy_point(&PQ2->P, &P);                    // B.P = P
copy_point(&PQ2->PmQ, &Q);                  // B.PmQ = Q
```

This means the three-point ladder computes $P + [m](P - Q)$, not $P + [m]Q$. The swap ensures the multiplied point avoids $(0, 0)$ for differential addition. This is not documented in the spec.
Advice for implementers: Undocumented conventions in reference code can look like bugs in your own code. Verify against the reference before "fixing" something that isn't broken.

4.7 Different formulas for the last two chain steps

The C reference uses three different formula variants across the $(2, 2)$-isogeny chain, controlled by two boolean flags `hadamard_bool_1` and `hadamard_bool_2`:

- **Normal steps** ($i < n-2$): `bool_1=0, bool_2=1`. Codomain is Hadamard-transformed (standard form). Eval computes $\mathcal{H}(\text{precomp} \cdot \mathcal{H}(P^2))$.
- **Penultimate step** ($i = n-2$): `bool_1=0, bool_2=0`. Codomain stays in dual form (no Hadamard). Eval computes $\text{precomp} \cdot \mathcal{H}(P^2)$.
- **Ultimate step** ($i = n-1$): `bool_1=1, bool_2=0`. Input gets an extra Hadamard before squaring; codomain in dual form. Eval computes $\text{precomp} \cdot \mathcal{H}(\mathcal{H}(P)^2)$.

The splitting step expects its input in dual form—what `bool_2=0` produces. Our code used the normal-step formula (`bool_2=1`) for all steps, feeding the splitting a Hadamard-transformed null point that has no zero $U_{i,j}(0)$ entry. The chain ran correctly for 125 steps and produced nonsense at the end.

Advice for implementers: The last few steps of an isogeny chain often need special handling. The spec describes the generic step uniformly; the `hadamard_bool` optimization is an implementation-level choice in the C reference that avoids redundant Hadamard transforms at the boundary between the chain and the splitting step. Check the loop structure, not just the per-step formula.

4.8 $\mathbb{F}_{p^2}$ square root canonicalization

This was the single most impactful bug we encountered, and we devote extra space to it because its consequences are subtle and far-reaching.

Every nonzero square in $\mathbb{F}_{p^2}$ has exactly two roots, r and $-r$. The SQIsign specification (Algorithm 8.12, the `difference_point` subroutine of `TorsionBasisFromHint`) says "compute a square root" without specifying which one. We initially returned whichever root the Tonelli–Shanks formula produced.

The C reference's `fp2_sqrt` (described in Aardal et al. [AAC+24], §3.1) has a five-line canonicalization step at the end: it negates the output whenever the real part is odd (as an integer in $[0, p)$), or when the real part is zero and the imaginary part is odd. This ensures a unique, deterministic "even" root for every input.

Without this canonicalization, `projective_difference` nondeterministically adds $+\sqrt{\Delta}$ or $-\sqrt{\Delta}$ to B_{XZ}, producing two affinely distinct candidates for $x(P-Q)$. Only one matches the C reference's `difference_point`. The wrong choice propagates through `ladder3pt` (producing a different kernel generator), the challenge isogeny (landing on a different curve E_{chl} with a different j-invariant), and every subsequent step. In our case, the challenge curve's j-invariant was `0x03007c`... instead of the correct `0x026558`...—completely unrelated values, not just a sign flip.

The key insight is that the choice of square root is *not absorbed by projective equivalence*. The formula $x_{P-Q} = (\sqrt{\Delta} + B_{XZ} \ : \ B_{ZZ})$ is not projectively equivalent to $(-\sqrt{\Delta} + B_{XZ} \ : \ B_{ZZ})$; these are genuinely different points. Any implementation of SQIsign that computes torsion bases via `difference_point` must canonicalize its $\mathbb{F}_{p^2}$ square root.

Advice for implementers: Any time a specification says "compute a square root" in $\mathbb{F}_{p^2}$, the implementation *must* specify and enforce a canonical choice. The even-real-part convention used by the C reference is simple and should be the default for any SQIsign implementation. This is not documented in the SQIsign specification as of v2.0.1.

4.9 Never recompute $P-Q$ via `DifferencePoint`

A torsion basis $(P, Q, P-Q)$ is produced once by `TorsionBasisFromHint`. The third component $P-Q$ is computed via `DifferencePoint`, which takes an $\mathbb{F}_{p^2}$ square root. We discovered—after fixing the sqrt canonicalization—that $P-Q$ must *never* be recomputed from P and Q in any subsequent operation. Even with a canonical sqrt, the two roots of

`DifferencePoint` yield the affinely distinct points $x(P-Q)$ and $x(2P-Q)$, and there is no guarantee that a fresh call returns the same one.

We found three separate sites where recomputation caused failure:

1. **After scaling.** We doubled P and Q to reduce their order, then called `projective_difference` on the doubled points instead of doubling the original $P-Q$ alongside.
2. **In the matrix application.** $M_{\text{chl}} \cdot (P, Q)$ produced P' and Q' via two biladder calls. We computed $P'-Q'$ via `projective_difference` instead of a third biladder with scalars $(a-b, c-d)$, as the C reference does in `matrix_scalar_application_even_basis`.
3. **Before the $(2,2)$-chain.** After the even response isogeny, we recomputed $P-Q$ from the images instead of pushing the original $P-Q$ through the isogeny as a third evaluation point.

Each fix was individually necessary; all three together were sufficient to make KAT vector 0 verify.

Advice for implementers: In x-only Montgomery arithmetic, $P-Q$ is a *derived value* that cannot be reconstructed from $x(P)$ and $x(Q)$ alone (because $x(P-Q) = x(P+Q)$ on the Kummer line, and `DifferencePoint` resolves this ambiguity via a square root whose sign is fragile). Treat $P-Q$ as a first-class member of the basis triple and propagate it through every transformation.

5 Bugs from Fixed-Width Arithmetic

The C reference uses GMP, so its BigInts grow as needed. We use a fixed-width `BigInt<N>` type with N 64-bit limbs, sized per Kim et al. [KLKL25] for NIST-I worst-case bounds. Fixed widths give us constant-time execution and stack allocation, but they add a new failure mode: *silent overflow*. `ct_mul` at width N returns a `BigInt<N>` and drops any bits above limb N. If the caller doesn't check, later code sees a wrapped value and produces wrong answers with no warning.

5.1 `pow_mod` silently overflows for large moduli

`pow_mod` does modular exponentiation by square-and-multiply:

```
pub fn pow_mod(base: &Self, exp: &Self, modulus: &Self) -> Self {
    let mut result = Self::ONE;
    for bit in exp.bits().rev() {
        result = result.ct_mul(&result).ct_mod(modulus);
        if bit { result = result.ct_mul(base).ct_mod(modulus); }
    }
    result
}
```

After each reduction, `result` $<$ `modulus`. But before the reduction, the product `result` $\cdot$ `result` can be as large as (`modulus` $-1)^2$. For this to fit without truncation, we need

$$64N \geq 2 \cdot \text{bits}(\texttt{modulus}).$$

Our commitment modulus $D_{\text{mix}} = 2^{512} + 75$ is 513 bits. Squaring needs 1026 bits, or about 17 limbs. The natural storage is `BigInt<9>` (576 bits), which is one limb above the bit-length. Called at that width, `pow_mod` truncates every squaring to 576 bits and returns garbage.

The bug is deceptive because `is_probable_prime`, `legendre`, and `modular_sqrt` all call `pow_mod`. One wrong arithmetic result at the bottom makes all three lie:

- `is_probable_prime(D_mix)` returns `false`, even though D_{mix} is prime.
- `legendre(4, D_mix)` returns -1, even though 4 is a perfect square (so the answer must be 1).
- `modular_sqrt` never finds a root. `RandomIdealGivenNorm` rejects every sample. Key generation hangs.

It stayed hidden because earlier tests only called these functions at `BigInt<4>` with inputs under 128 bits—small enough that the squares fit. It surfaced the first time we ran `SigningKey::generate` and evaluated Euler's criterion at D_{mix} scale.

Immediate fix. In `random_prime_norm_wide`, widen the inputs to `BigInt<18>` (1152 bits) before the Legendre check and square root, then narrow the result back to `BigInt<9>`. 18 is the smallest width where $64N \geq 2 \cdot 513$, with room to spare.

Proper fix (tracked). `pow_mod` and its three callers should take a method-level const generic W for the working width, separate from the storage width N. Callers pass $W \geq 2N$ explicitly, enforced by a `const` assertion. The current single-width entry points still work for small moduli (under $32N$ bits), where N is already enough. Until we do this, every call site that passes a near-full modulus is potentially wrong and will fail silently.
Advice for implementers: With fixed-width arithmetic, every call site that chains multiplications needs a width budget. A function that takes a `BigInt<N>` is not self-contained: its correctness depends on N being big enough for its internal operations. Document the requirement in the rustdoc, and enforce it at compile time with const generics or a sealed trait when it's nontrivial. The worst bugs are the ones where a wrong value at the bottom of the stack flips a cryptographic predicate—"is this prime?", "is this a square?"— with no other symptom.

5.2 Five bugs in `GeneralizedRepresentInteger`

Algorithm 3.12 (GeneralizedRepresentInteger) finds $\gamma \in \mathcal{O}$ with $\text{nrd}(\gamma) = M$ by sampling (z, t) and solving $x^2 + qy^2 = M'$ via Cornacchia. Our implementation had five independent errors that together prevented the algorithm from ever succeeding during signing:

1. **Off-by-one in z sampling.** The spec says "sample z uniformly from $[1, \ldots, m]$" (line 4). Our loop variable started at 2 instead of 1, skipping the crucial $z = 1$ case. With z even, $M' = 4M - p(z^2 + qt^2)$ is always even and therefore never prime. The `is_probable_prime` check rejected every candidate, making the algorithm appear to find no solutions at all.

2. **t sampled from $[0, m']$ instead of $[-m', m']$.** The spec says "sample t uniformly from $[-m', \ldots, m']$" (line 5). We sampled only non-negative values. Since M' depends on t^2, the Cornacchia output (x, y) is the same for $\pm t$, but the isogeny condition (line 15, $x - t \equiv 2 \pmod 4$) depends on the sign of t. Missing negative t values halves the chance of satisfying this congruence.

3. **Incorrect divisibility-by-2 check.** The spec constructs $\gamma = x + \omega y + jz + \omega jt$ and checks that the largest d with $\gamma/d \in \mathcal{O}$ equals 2 (lines 18–19). We initially replaced this with an "all coordinates same parity" check in $\{1, i, j, k\}$, then with a hand-derived congruence condition. Both were wrong.

   The correct approach, matching the C reference's `quat_alg_make_primitive`, is to construct γ as a quaternion element in the order, compute the GCD of its coordinates

in the order's integral basis, and verify the GCD equals 2. Attempting to reason about divisibility-by-2 in the $\{1, i, j, k\}$ coordinates is fragile because the order $\mathcal{O}_0$ has a basis with half-integer entries ($\frac{1+j}{2}$, $\frac{i+k}{2}$); divisibility by 2 in the order is not equivalent to any simple parity condition on the $\{1, i, j, k\}$ coordinates.

4. **Arithmetic overflow in primality and Cornacchia.** The candidates M' are ∼273-bit values stored in `BigInt<8>` (512 bits). Both `is_probable_prime` and `cornacchia` internally call `pow_mod`, whose squaring step produces ∼546-bit intermediates that overflow the 512-bit storage. The result is that `is_probable_prime` rejects genuine primes and `cornacchia` returns `None` for valid inputs. Fixed by using the widened variants `is_probable_prime_w::<9>` and `cornacchia_w::<9>` (576 bits $\geq 2 \times 273$).

5. **Hardcoded search bound too small.** We initially used `bound = 256`. The spec computes the bound as $\lceil \sqrt{4M/(p\sqrt{q})} \rceil$, which for NIST-I at the FixedDegreeIsogeny call site is ∼1400–2500. With 256 iterations, the search exhausted without finding any (z, t, x, y) satisfying both the isogeny condition and the $d = 2$ condition. Fixed by computing the bound from the spec's formula.

Bugs 1 and 4 were the most damaging: bug 1 eliminated all candidates before primality testing, and bug 4 made the primality test reject genuine primes. Bug 3 was the subtlest and required the most iterations to diagnose, since the divisibility condition depends on the order's algebraic structure rather than a simple coordinate check.

Advice for implementers: Algorithm 3.12 is simple at first glance—a short loop with Cornacchia and a divisibility check—but the implementation details (sampling ranges, sign conventions, order-basis divisibility, arithmetic width, and search bounds) are all places where a plausible-looking implementation silently fails. The divisibility check in particular should not be hand-derived from coordinate arithmetic; it should use the same order-basis GCD computation as the C reference.

5.3 Invalid test kernels for $(2, 2)$-isogeny chains

This was the most time-consuming false lead we encountered. We wrote three unit tests that exercised the $(2, 2)$-isogeny chain on short chains ($e = 2, 3$) with synthetic kernel points constructed from arbitrary basis elements — for example, $(P, Q) \times (Q, P)$ on $E_0 \times E_0$, where P, Q are torsion basis points. All three consistently failed at the splitting step (Algorithm 8.41), which found zero vanishing $U_{i,j}(0)$ coordinates instead of the expected one.

Because 100 KAT verification vectors exercised the same chain code and passed, we assumed the test data was valid and spent considerable effort searching for a bug in our implementation: comparing every intermediate value against the C reference, tracing Hadamard transform conventions through the chain, checking projective representatives from Jacobian doubling, and adding isotropicity diagnostics. None of these investigations found a discrepancy — the formulas matched the C reference exactly.

We had originally pulled these kernel constructions from the C reference implementation's test suite without modification or independent validation. Because the C reference exercises the $(2, 2)$-chain only through the full signing/verification flow (which produces valid kernels by construction), its tests never exercise synthetic kernels of this form. We adopted the kernel patterns without checking whether they satisfied the theta formula preconditions.

The test data was invalid. The synthetic kernels are *isotropic* in the Weil-pairing sense (the product Weil pairing is trivial), but they cause the `ActionByTranslation` determinant $\delta = x_{P_4} z_{P_2} - z_{P_4} x_{P_2}$ to vanish. The theta change-of-basis (Algorithm 8.36) requires inverting δ; when $\delta = 0$ the formulas silently produce wrong output. We confirmed this by

running the same kernels through the DMPR24 reference SageMath implementation [**?**], which also fails with a `ZeroDivisionError` in the same batched inversion.

The specification does not document the nonvanishing-δ precondition. The C reference avoids it in practice because its kernels come from FixedDegreeIsogeny (Algorithm 3.15), which constructs kernels from endomorphisms produced by RepresentInteger — these structured kernels have nonzero δ by construction. When δ does vanish (a rare edge case), the C reference's `gluing_compute` returns failure and the signing loop retries.

Advice for implementers: Not every isotropic subgroup of $E_1[2^e] \times E_2[2^e]$ is a valid kernel for the theta $(2,2)$-isogeny formulas. The formulas additionally require nonvanishing of certain determinants in the theta change-of-basis computation. This precondition is nowhere stated in the specification and is not obvious from the pseudocode. We wasted significant debugging time because our test data violated an undocumented assumption. The fix was to delete the invalid tests and rely on KAT verification (100 vectors, all passing), which exercises the chain on kernels produced by the actual protocol flow.

5.4 HNF coefficient blow-up at fixed precision

The commitment phase constructs the left ideal $I = O_0\langle\gamma, N\rangle$ by concatenating the eight column vectors of $O_0 \cdot \gamma$ and $O_0 \cdot N$ and reducing to Hermite Normal Form (Algorithm 3.2). The C reference uses GMP, so HNF intermediate entries grow without bound. We use fixed-width `BigInt<30>` (1920 bits).

Classical HNF's extended-GCD elimination produces intermediate column entries whose magnitude can grow exponentially in the rank. For our rank-4 ideal lattices with initial entries up to $\sim 2^{770}$ (the $p \cdot g_i$ products from `RandomIdealGivenNorm`), intermediate xgcd products exceed the 1920-bit budget after only a few elimination steps. The overflowing limbs are silently truncated, and the resulting "HNF" basis collapses: we observed basis columns with reduced norms of 4 and $\sim 2^{251}$—elements of the ambient order O_0, not the intended ideal. Every downstream operation then computes on a lattice unrelated to I, and `reduce_to_prime_norm` never finds a valid prime because the divisibility invariant $\mathrm{nrd}(\alpha) \in N(I) \cdot \mathbb{Z}$ no longer holds.

The fix is the standard Domich–Kannan–Trotter modular HNF (Cohen, *A Course in Computational Algebraic Number Theory*, §2.4.2). If D is any positive multiple of the lattice determinant $\det(L)$, then $L \supseteq D \cdot \mathbb{Z}^n$, so reducing every intermediate entry modulo D after every arithmetic update preserves the HNF while bounding entries by D.

For the commitment ideal at NIST-I, $D = 4d^4N^2p \approx 2^{1282}$ (21 limbs) is a precomputed constant. The modular HNF operates at a wider intermediate width ($W = 44$ limbs, 2816 bits) to hold products of two D-sized values before reduction, then narrows the result back to the storage width. The same technique is applied to two additional HNF sites:

- **Response-phase ideal construction** (`from_generator` at width 30): the generator α_{rsp} from the sampling step has coordinates up to $\sim 2^{1400}$ bits; the modulus is computed per call as $4d^4 \cdot (norm)^2 \cdot p$.
- **Post-reduce basis update** inside `reduce_to_prime_norm`: the new basis columns are quaternion products with entries up to $\sim 2^{900}$ bits and integer-column covolume up to $\sim 2^{3000}$ bits. The modulus and intermediate arithmetic are widened to `BigInt<50>` and `BigInt<100>` respectively.

Advice for implementers: Algorithm 3.2 of the SQIsign specification describes HNF over arbitrary-precision integers; it does not discuss fixed-precision adaptations or coefficient-size bounds. Any fixed-width implementation of Algorithm 3.2 must either prove that intermediate growth stays within the budget for the specific input magnitudes, or use modular HNF with a known multiple of the lattice determinant as the bounding modulus. The C reference sidesteps this entirely by using GMP.

5.5 Missing u^{-1} normalization in FixedDegreeIsogeny

Algorithm 3.15 (FixedDegreeIsogeny) computes $\theta \leftarrow$ RepresentInteger$(u \cdot (2^e - u), \mathcal{O}_t, \text{true})$ on line 2, then constructs a $(2,2)$-isogeny kernel from $([u]P, \theta(P))$ and $([u]Q, \theta(Q))$ on line 4. Since $\text{nrd}(\theta) = u \cdot (2^e - u)$, the endomorphism θ has degree $u \cdot (2^e - u)$, and the actual kernel of the degree-$(2^e - u)$ isogeny we want is generated by $(P, (\theta/u)(P))$—not by $([u]P, \theta(P))$.

The spec's pseudocode says to form the kernel from $([u]P, \theta(P))$ and double $f - 2 - e$ times. This is algebraically equivalent to using $(P, (\theta/u)(P))$ only if the u-torsion is killed by the doublings, which it is. However, in practice, applying θ directly (without dividing by u) produces kernel points whose ActionByTranslation determinant is degenerate. Every one of the hundreds of $(2,2)$-chains we attempted failed with either zero or ten vanishing $U_{i,j}(0)$ indices (the correct count for a valid product kernel is exactly one).

The C reference (`dim2id2iso.c`, lines 133–143) multiplies all four quaternion coordinates of θ by $u^{-1} \bmod 2^{e+2}$ before applying the endomorphism to the torsion basis:

```
ibz_mul(&two_pow, &two_pow, &ibz_const_two);
ibz_mul(&two_pow, &two_pow, &ibz_const_two);
ibz_invmod(&tmp, u, &two_pow);
ibz_mul(&theta.coord[0], &theta.coord[0], &tmp);
ibz_mul(&theta.coord[1], &theta.coord[1], &tmp);
ibz_mul(&theta.coord[2], &theta.coord[2], &tmp);
ibz_mul(&theta.coord[3], &theta.coord[3], &tmp);
```

This effectively applies θ/u (mod the torsion order) to the basis, so the kernel is $(P, (\theta/u)(P))$ as intended. The modulus 2^{e+2} suffices because the kernel lives in $E[2^e]$ and the extra $+2$ accounts for the gluing's order-4 structure. Without this step, the inner $(2,2)$-chain *always* fails.

Advice for implementers: Algorithm 3.15 should either (a) state that the kernel is formed from $(P, (\theta/u)(P))$ and explain the modular inversion, or (b) prove that the $([u]P, \theta(P))$ formulation produces a non-degenerate kernel without normalization. The current pseudocode silently requires a step that is absent from the algorithm box.

5.6 Sign-magnitude to unsigned modular conversion

The quaternion algebra's `decompose` function returns coefficients as signed integers (e.g., $c_0 = -8$ for $\theta = 3 + 5i + 7j + 11k$ in $\mathcal{O}_0$). These coefficients are used as scalars in the biladder to compute $\theta(P) = c_0 \cdot P + c_1 \cdot M_{\text{gen2}} + \cdots$ on the torsion basis $E_t[2^f]$.

Our `BigInt` type uses sign-magnitude representation, while `Scalar` (the biladder's input) is unsigned modular ($\bmod 2^{256}$). The `From<BigInt> for Scalar` conversion originally took the raw limbs without checking the sign bit, mapping -8 to $+8$ instead of $2^{256} - 8$. This produced the wrong endomorphism image for *every* nontrivial quaternion element from RepresentInteger, while passing tests that used simple elements like i or scalar multiples (whose decomposition has no negative coefficients).

Advice for implementers: Every boundary between signed and unsigned integer representations is a potential sign-swallowing bug. The type system did not prevent this because both `BigInt` and `Scalar` store the same `[u64; 4]` limbs — only the interpretation differs.

6 Type Safety for Cryptographic Invariants

Rust's type system lets us make illegal states unrepresentable. Each of the following types prevents a class of bug that would compile silently in C.

6.1 IsogenyDegree

Isogeny degrees in SQIsign are always positive odd integers bounded by 2^{f-2} (246 bits for NIST-I). In the C reference, they are bare `ibz_t` values—the same type used for everything else. Nothing prevents passing an even number or a negative value.

We represent degrees as `IsogenyDegree([u64; 4])`—unsigned, positive by construction, oddness enforced by the only constructor:

```
pub fn new_odd(limbs: [u64; 4]) -> Option<Self> {
    if limbs[0] & 1 == 0 { return None; }
    Some(Self(limbs))
}
```

Passing an even number to `FixedDegreeIsogeny` is a type error.

6.2 TorsionExponent

Torsion exponents (e such that 2^e divides the group order) are bounded by $f = 248$. A bare `u32` allows 4 billion values; only 249 are valid. `TorsionExponent(u32)` with a range check in the constructor catches this at the API boundary. Every function that takes a torsion exponent—`action_matrix`, `to_kernel`, `fixed_degree_isogeny`—uses this type, not `u32`.

6.3 HnfLattice vs. Lattice

Lattice containment checking requires Hermite Normal Form. Calling it on a non-HNF lattice silently produces wrong answers. We use two types: `Lattice<N>` (arbitrary basis) and `HnfLattice<N>` (guaranteed HNF, constructed only via `From<Lattice>`). The `contains()` method exists only on `HnfLattice`. You cannot call it on the wrong type.

6.4 IdealFactor

`SuitableIdeals` returns two factors, each with an element β, a degree d, and a parent order. In the C reference, these are six separate variables. Accidentally pairing one factor's degree with the other's element is a plausible mistake—the types are the same. Our `IdealFactor` struct bundles them:

```
pub struct IdealFactor {
    pub order: &'static ExtremalOrder<4>,
    pub beta: Element,
    pub degree: IsogenyDegree,
}
```

Cross-wiring requires reaching into both structs and swapping fields deliberately—not something you do by accident.

7 Constant-Time Considerations

7.1 The spec's silence on side channels

The SQIsign specification analyzes `SuitableIdeals` only in terms of failure probability (§9.3.2), not side-channel resistance. By tracing Algorithm 4.2 (signing), we confirmed that the quaternion algorithms operate on data derived from the secret key I_{sk} (lines 13–19). The variable-time behavior of the C reference is a pragmatic gap, not a deliberate security argument.

7.2 Published CT techniques

Seven papers form a complete constant-time stack for SQIsign v2:

- Kouider et al. [KMJK25] provide constant-time big integer arithmetic (division, exponentiation, square root) up to ∼12,000 bits.
- Hanyecz et al. [HKK+25] give the first constant-time LLL-like lattice reduction for dimension 4, replacing the variable-time L2 algorithm used in `SuitableIdeals`.
- Basso et al. [BHJ+25] cover CT HNF, CT `IdealGenerator`, and CT `RepresentInteger`—the three core quaternion subroutines.
- Kim et al. [KLKL25] prove explicit worst-case size bounds for all quaternion intermediates in Round-2 SQIsign, enabling fixed-precision (no multi-precision) arithmetic.
- Mukherjee et al. [MCJ+25] and Jacquemin et al. [JMKR23] demonstrate and propose mitigations for SPA attacks on Cornacchia's algorithm—the most urgent constant-time target.

7.3 Current status

Signing and key generation are **not yet constant-time.** The entire quaternion arithmetic layer—lattice reduction (L2/LLL), Hermite Normal Form, Cornacchia/RepresentInteger, and IdealToIsogeny—is variable-time on secret-derived data. Every such code path is annotated with `TODO(ct)` citing the Algorithm 4.2 line that makes the input secret-derived.

Field arithmetic (§??), curve operations, and verification are constant-time or operate on purely public data.

We plan a six-phase CT hardening immediately after achieving end-to-end interoperability.

8 Test Infrastructure

Beyond the 100 KAT vectors from the C reference implementation (all of which verify successfully), we maintain a growing suite of negative and vulnerability test vectors in the C2SP/wycheproof JSON format [BDK+24]. These vectors are designed to detect not just bugs but *vulnerabilities*—they exercise rejection paths that the KAT vectors, being all valid, never touch.

Invalid vectors are independently generated by byte-level perturbation (no dependency on the C reference implementation). Valid vectors cite the C reference KAT file as their provenance; we intend to replace them with independently Sage-generated vectors for eventual upstream contribution to the C2SP/wycheproof project.

8.1 Algorithm naming

We identify the parameter set as `SQIsign_248`, where 248 is the torsion exponent f in $p = 5 \cdot 2^{248} - 1$. The exponent is the fundamental parameter from which all others derive: field size, isogeny degrees, key sizes, signature sizes. Unlike "NIST-I" (a security-level label that could be reassigned to different parameters) or "353" (a derived quantity—the secret key size in bytes—that would change if the encoding changes), $f = 248$ pins the actual cryptographic parameters.

8.2 Verification vectors

Table 1 summarizes the verification vector categories. Each vector consists of a public key, message, signature, and expected result (`valid` or `invalid`). Invalid vectors are derived from valid KAT vectors by targeted perturbation.

Table 1: Verification test vector categories for `SQIsign_248` (C2SP/wycheproof JSON format).

Category	Count	What it detects
Normal (valid KAT)	2	Baseline: C ref signatures verify.
WrongMessage	2	Different, empty, extended, and truncated messages.
WrongPublicKey	1	Cross-key verification.
BitFlip (per field)	7	Single-bit corruption in each of the seven signature fields.
AllZero / AllOnes	2	Degenerate byte patterns.
ExponentOverflow	4	$n_{\text{bt}} + r_{\text{rsp}} > e_{\text{rsp}}$, causing e'_{rsp} underflow.
NonCanonicalFp	3	Field elements $\geq p$ in `E_aux` or pk. Silently reduces mod p, producing the wrong curve.
SingularCurve	3	$A = \pm 2$ (singular Montgomery curve, discriminant zero) in `E_aux` or pk.
SpecialJInvariant	1	$A = 0$ ($j = 1728$, extra automorphisms $\pm 1, \pm i$).
StartingCurve	1	`E_aux` $= E_0$ ($A = 6$).
DegenerateMatrix	2	`M_chl` all-zero or identity.
ZeroChallenge	1	Challenge $= 0$ (degenerate Fiat–Shamir).
ScalarOverflow	1	Matrix entry $\geq 2^{e_{\text{rsp}}}$.
ChallengeOverflow	1	High bits set beyond $e_{\text{chl}} = 122$.
SplicedField	3	Signature field replaced with the corresponding field from a different valid signature.
MalformedPk	2	All-zero or all-`0xFF` public key.
Total	**39**	

Finding: verification panics on malformed input. Several invalid vectors (bit flips in `E_aux`, `M_chl`, and `chl`) reach the $(2, 2)$-isogeny splitting step (Algorithm 8.42, GetIndexSplitting), which uses a `debug_assert!` to check that exactly one null-point index is zero. Malformed signatures violate this invariant, and the assertion panics rather than returning an error. In debug builds this is a denial-of-service vector: any untrusted signature can crash the verifier. In release builds the assertion compiles out and the function returns a wrong index silently. The correct fix is to return `Option<usize>` and propagate the error as `VerificationFailed`. Our test runner uses `std::panic::catch_unwind` as a temporary workaround.

8.3 Key generation vectors

We maintain 12 keygen/sk-parsing vectors:

- Deterministic keygen from KAT seeds produces the expected (pk, sk) byte-for-byte.
- The pk embedded in the first 65 bytes of sk matches the standalone pk.
- Malformed sk rejection: truncated, extended, all-zero, all-`0xFF`, single-bit flips in each region (pk, ideal norm, M_{sk} matrix), and a "Frankenstein" sk with the pk from one vector spliced onto the sk body of another.

8.4 Signing vectors (planned)

The signing vector file is a skeleton with one vector (C ref KAT round-trip) and a roadmap for categories to add as signing stabilizes:

- **NonceIndependence**: same (sk, msg) with different randomness must produce distinct commitment curves. Nonce reuse leaks the secret key.
- **CornacchiaBranching**: inputs crafted so that RepresentInteger's Cornacchia subroutine takes different branches, targeting SPA vulnerabilities [MCJ+25].
- **ResponseEdgeCases**: signatures with $r_{\text{rsp}} > 0$ or $n_{\text{bt}} > 0$, exercising the even-response isogeny and backtracking paths that are rare in KAT vectors.

8.5 Mutation testing

We use `cargo-mutants` [Poo24] to verify that the test suite catches bugs, not just that it runs. Mutation testing inserts faults (replacing operators, return values, conditionals) and checks whether any test fails. A surviving mutant is a missing test.

Running `cargo-mutants` on the `keys/` module (212 mutants) found 8 surviving mutants—6 in `ChallengeMatrix::from_bytes` validation and 2 in keygen—all of which represented real test gaps. Writing targeted tests killed the validation mutants; the keygen mutants survive because keygen tests are slow (each invocation runs a $(2, 2)$-isogeny chain) and are marked `#[ignore]`.

CI runs incremental mutation testing on every pull request (only changed code via `--in-diff`) and a full sharded run weekly.

9 Current Status and Future Work

Verification is complete: all KAT vectors from the C reference implementation verify successfully, exercising the full pipeline from torsion basis construction through the $(2, 2)$-isogeny chain and splitting.

Signing compiles end-to-end and is structurally complete. The commitment phase, challenge derivation, and response phase are wired together, with `Order<N>` tracking maximal orders through the Deuring correspondence. Two coupled blockers remain:

- The sampling radius in the response phase is a placeholder; the correct value $D_{\text{rsp}} \cdot D_{\text{mix}}^2 \cdot 2^{f+1} \approx 2^{1399}$ requires wider lattice arithmetic (∼22 limbs for the radius, ∼12 for lattice entries).
- Degree computations in the response phase are missing a D_{mix}^2 division, coupled with the radius fix.

Key generation (Algorithm 4.1) is not yet implemented. Signature serialization to the 148-byte wire format is implemented.

Signing and key generation are **not yet constant-time.** The quaternion arithmetic layer is variable-time on secret-derived data (see §7 for details and our hardening plan).

Next steps.

1. Widen the response-phase sampling and degree arithmetic to achieve a first successful signature;
2. Wire key generation;
3. Achieve full KAT interoperability (sign + verify round-trip);
4. Execute the six-phase CT hardening plan (§7).

References

[AAC+24] Martha Norberg Hovd Aardal, Maria Pearson Azimova, Efrén López Carrión, Lorenz Dillinger, and Matthias J. Kannwischer. Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4. Cryptology ePrint Archive, Paper 2024/1563, 2024. https://eprint.iacr.org/2024/1563.

[BDK+24] Daniel Bleichenbacher, Thai Duong, Emilia Käsper, Quan Nguyen, et al. Project Wycheproof: Test vectors for cryptographic libraries. https://github.com/C2SP/wycheproof, 2024.

[BHJ+25] Andrea Basso, Chenfeng He, David Jacquemin, Fatna Kouider, Péter Kutas, Anisha Mukherjee, Sina Schaeffler, and Sujoy Sinha Roy. Constant-Time Quaternion Algorithms for SQIsign. Cryptology ePrint Archive, Paper 2025/2192, 2025. https://eprint.iacr.org/2025/2192.

[HKK+25] Otto Hanyecz, Alexander Karenin, Elena Kirshanova, Péter Kutas, and Sina Schaeffler. Constant time lattice reduction in dimension 4 with application to SQIsign. Cryptology ePrint Archive, Report 2025/027, 2025. URL: https://eprint.iacr.org/2025/027.

[JMKR23] David Jacquemin, Anisha Mukherjee, Péter Kutas, and Sujoy SINHA ROY. Ready to SQI? Safety first! Towards a constant-time implementation of isogeny-based signature, SQIsign. Cryptology ePrint Archive, Report 2023/807, 2023. URL: https://eprint.iacr.org/2023/807.

[KLKL25] Won Kim, Jeonghwan Lee, Hyeonhak Kim, and Changmin Lee. SQIsign with fixed-precision integer arithmetic. Cryptology ePrint Archive, Report 2025/1649, 2025. URL: https://eprint.iacr.org/2025/1649.

[KMJK25] Fatna Kouider, Anisha Mukherjee, David Jacquemin, and Péter Kutas. Constant-time integer arithmetic for SQIsign. Cryptology ePrint Archive, Report 2025/832, 2025. URL: https://eprint.iacr.org/2025/832.

[MCJ+25] Anisha Mukherjee, Maciej Czuprynko, David Jacquemin, Péter Kutas, and Sujoy Sinha Roy. Simple power analysis attack on SQIsign. Cryptology ePrint Archive, Report 2025/830, 2025. URL: https://eprint.iacr.org/2025/830.

[Poo24] Martin Pool. `cargo-mutants`: Mutation testing for Rust. https://mutants.rs/, 2024.

[Rus24] Rust team. Rust API Guidelines. https://rust-lang.github.io/api-guidelines/, 2024.

[SQI25] SQIsign team. SQIsign: Compact Post-Quantum Signatures from Quaternions and Isogenies. Specification version 2025-07-07, 2025. https://sqisign.org/spec/sqisign-20250707.pdf.